SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	**Assessment:**	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	30% of CIA	**Total Marks:**	100
CO5 Statement:	*Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.*		
Student Name:	Tejaswi Reddy K	**Reg. No.:**	192371036
Section / Batch:	CS-BIOSCI	**Date:**	01-09-2026

Instructions to Students

- Answer ALL questions with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems.
- Neat presentation and step-by-step justification are mandatory.

Question Type	Focus Area	Questions	Marks
Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1 – Q3	Q1 –40 Q2 –30 Q3 -30
	GRAND TOTAL:	**100 Marks**	

SECTION A – Pseudocode Writing Task

Q1. N-Queens Problem Design a pseudocode using Backtracking to solve the N-Queens problem for an N×N chessboard. Clearly identify inputs (board size) and expected output (valid queen placements). Develop a logically correct algorithm that ensures no two queens attack each other. Use appropriate control structures such as loops and conditional checks for row, column, and diagonal safety. Ensure the pseudocode is well-structured and easy to follow. Handle all possible configurations and optimize by pruning invalid placements.

SOLUTION –

The **N-Queens problem** is the problem of placing **N queens on an N × N chessboard** such that no two queens attack each other.

A queen can attack another queen if they are in the same:

- **Row**
- **Column**
- **Main diagonal**
- **Anti-diagonal**

The backtracking approach places queens **one row at a time** and immediately rejects a placement if it is unsafe.

Pseudocode

```
Algorithm N-Queens(N)

Input:
    N – Size of the chessboard (N × N)

Output:
    A valid arrangement of N queens
    or "No solution exists"

   Create an N × N board and initialize all cells to EMPTY

   if PlaceQueens(board, 0, N) = TRUE then
   Print board
   else
   Print "No solution exists"
   end if


   Procedure PlaceQueens(board, row, N)

   if row = N then
   return TRUE
   end if

   for col ← 0 to N – 1 do

   if IsSafe(board, row, col, N) = TRUE then

   board[row][col] ← QUEEN

   if PlaceQueens(board, row + 1, N) = TRUE then
   return TRUE
   end if

   board[row][col] ← EMPTY
   // Backtrack and try the next column

   end if

   end for

   return FALSE


   Procedure IsSafe(board, row, col, N)

   // Check the same column
   for i ← 0 to row – 1 do
   if board[i][col] = QUEEN then
   return FALSE
   end if
   end for
```

```
    // Check upper-left diagonal
    i ← row − 1
    j ← col − 1

    while i ≥ 0 AND j ≥ 0 do
    if board[i][j] = QUEEN then
    return FALSE
    end if
    i ← i − 1
    j ← j − 1
    end while

    // Check upper-right diagonal
    i ← row − 1
    j ← col + 1

    while i ≥ 0 AND j < N do
    if board[i][j] = QUEEN then
    return FALSE
    end if
    i ← i − 1
    j ← j + 1
    end while

    return TRUE
```

How the Algorithm Works

The algorithm follows **backtracking**:

1. Start from the **first row**.
2. Try placing a queen in each column.
3. Before placing it, call `IsSafe()` to check:
    - Column
    - Upper-left diagonal
    - Upper-right diagonal
4. If the position is safe, place the queen.
5. Move to the **next row** using recursion.
6. If no position is possible in the next row:
    - Remove the previously placed queen.
    - Try another column.
7. Continue until queens are successfully placed in all NN rows.

This removal and retry operation is called **backtracking**.

Example: N = 4

One valid solution is:

```
. Q . .
. . . Q
Q . . .
. . Q .
```

Here:

- Row 1 → Column 2
- Row 2 → Column 4
- Row 3 → Column 1
- Row 4 → Column 3

The queens do not share the same row, column, or diagonal.

Therefore, this is a valid solution:

[2,4,1,3][2,4,1,3]

Pruning Invalid Placements

The important optimization is that the algorithm **does not continue exploring a placement once it is known to be unsafe**.

For example:

```
if IsSafe(board, row, col, N) = TRUE
```

Only safe positions are explored.

If a queen attacks another queen, that branch of the search tree is immediately abandoned:

```
if unsafe then
    do not place queen
```

This significantly reduces unnecessary combinations compared with checking every possible arrangement after placing all queens.

Why the Algorithm Is Correct

The algorithm guarantees a valid solution because:

- **One queen is placed per row**, so no two queens share a row.
- Before placing a queen, its **column is checked**, so no two queens share a column.
- Both **diagonals are checked**, so no two queens can attack diagonally.
- If a partial arrangement cannot lead to a solution, the algorithm **backtracks** and tries another possibility.
- It explores all possible placements that are not pruned as unsafe.

Thus, if a solution exists, the algorithm will find a valid arrangement.

Q2. Subset Sum Problem Write a pseudocode to determine whether a subset of given integers sums to a target value using Backtracking. Clearly define inputs (set of numbers and target sum) and output (true/false or subset). Design a complete algorithm with proper branching and pruning conditions. Use control structures effectively to explore subsets. Present the pseudocode in a structured and readable format. Ensure the solution handles all cases and avoids unnecessary computations.

SOLUTION –

The **Subset Sum Problem** asks whether there exists a subset of a given set of integers whose sum is exactly equal to a specified target value.

The backtracking approach explores two choices for every element:

1. **Include** the current element in the subset.
2. **Exclude** the current element from the subset.

Invalid branches are pruned whenever it is clear that they cannot produce the target sum.

Pseudocode

```
Algorithm Subset-Sum(A, N, Target)

Input:
    A[0 ... N-1] – Array of N integers
    Target       – Required target sum

Output:
    TRUE and the subset whose sum is Target
    OR
    FALSE if no such subset exists

   Subset ← empty set

   if FindSubset(A, 0, N, 0, Target, Subset) = TRUE then
   Print "Subset found:"
   Print Subset
   return TRUE
   else
   Print "No subset exists"
   return FALSE
   end if


   Procedure FindSubset(A, index, N, CurrentSum, Target, Subset)

   // Base condition
   if CurrentSum = Target then
   return TRUE
   end if

   // No elements left
   if index = N then
   return FALSE
   end if

   // Pruning condition
```

```
if CurrentSum > Target AND
      all remaining numbers are non-negative then
return FALSE
end if

// Branch 1: Include A[index]
Add A[index] to Subset

if FindSubset(A, index + 1, N,
                  CurrentSum + A[index],
                  Target, Subset) = TRUE then
return TRUE
end if

// Backtrack: remove A[index]
Remove A[index] from Subset

// Branch 2: Exclude A[index]
if FindSubset(A, index + 1, N,
                  CurrentSum,
                  Target, Subset) = TRUE then
return TRUE
end if

return FALSE
```

How the Algorithm Works

Suppose:

A={3,4,5,2}A=\{3,4,5,2\}

and

Target=9

The algorithm starts with an empty subset:

{}, CurrentSum=0\{\}, \quad CurrentSum=0

For every element, it creates two possibilities.

For element 3

Include 3:

{3},Sum=3\{3\},\quad Sum=3

Then consider 4.

Include 4:

{3,4},Sum=7\{3,4\},\quad Sum=7

Then consider 5.

Include 5:

{3,4,5},Sum=12\{3,4,5\},\quad Sum=12

Since the sum has exceeded the target and all numbers are non-negative, this branch is **pruned**.

The algorithm backtracks and tries other possibilities.

Eventually it finds:

{3,4,2}\boxed{\{3,4,2\}}

because:

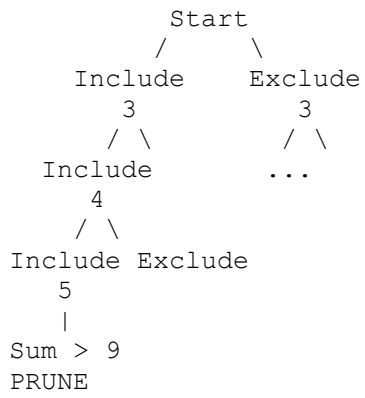
3+4+2=93+4+2=9

Therefore, the output is:

```
Subset found:
{3, 4, 2}
TRUE
```

Decision Tree Concept

For each element, the algorithm branches into **Include** and **Exclude**:



This continues until either:

- `CurrentSum = Target` → **Solution found**
- All elements are exhausted → **No solution in that branch**

Backtracking

Backtracking occurs when a selected element does not lead to a solution.

For example:

```
Subset = {3, 4, 5}
Sum = 12
```

Since this cannot produce 9 using only non-negative remaining numbers, the algorithm removes 5:

```
Subset = {3, 4}
Sum = 7
```

and tries the next possibility.

This avoids exploring unnecessary branches.

Pruning Conditions

The main pruning condition is:

```
if CurrentSum > Target
```

when **all numbers are non-negative**.

For example:

CurrentSum=12, Target=9CurrentSum=12,\ Target=9

There is no need to continue that branch because adding more non-negative numbers can never reduce 12 to 9.

Important Note for Negative Numbers

If the input can contain **negative integers**, the simple `CurrentSum > Target` pruning rule is **not valid**.

For example:

CurrentSum=12, Target=9

A remaining value of −3-3 could still produce:

12+(−3)=912+(-3)=9

Therefore, for arbitrary integers, the algorithm should **not use this pruning condition** unless a valid bound based on the remaining positive and negative values is calculated.

A fully general version can simply remove lines 9–12 and still correctly solve the problem.

Q3. Graph Coloring Problem Develop a pseudocode using Backtracking to assign colors to vertices of a graph such that no two adjacent vertices share the same color. Clearly identify inputs (graph, number of colors) and output (valid coloring). Design a correct and efficient algorithm using loops and condition checks. Ensure proper use of recursion and validation functions. Structure the pseudocode clearly and logically. Handle all constraints and ensure completeness of the solution.

SOLUTION –

The **Graph Coloring Problem** is to assign colors to all vertices of a graph such that **no two adjacent vertices have the same color**.

The backtracking algorithm tries to assign a color to each vertex one at a time. If a color creates a conflict with an already-colored adjacent vertex, that color is rejected. If no color can be assigned, the algorithm **backtracks** to the previous vertex and tries another color.

Pseudocode

```
Algorithm Graph-Coloring(G, M)

Input:
    G = (V, E) – Graph with N vertices
    M          – Number of available colors

Output:
    A valid coloring of all vertices
    or "No solution exists"

   N ← number of vertices in G

   Create array Color[0 ... N-1]
   Initialize all Color[i] ← 0
   // 0 means no color assigned

   if Color-Graph(G, M, Color, 0, N) = TRUE then
   Print "Valid coloring:"
   for i ← 0 to N – 1 do
   Print "Vertex", i, "→ Color", Color[i]
   end for
   else
   Print "No valid coloring exists"
   end if


   Procedure Color-Graph(G, M, Color, Vertex, N)

   // Base condition
   if Vertex = N then
   return TRUE
```

```
end if

// Try every available color
for C ← 1 to M do

if Is-Safe(G, Color, Vertex, C, N) = TRUE then

// Assign color
Color[Vertex] ← C

// Recursively color the next vertex
if Color-Graph(G, M, Color,
                        Vertex + 1, N) = TRUE then
return TRUE
end if

// Backtrack
Color[Vertex] ← 0

end if

end for

return FALSE


Procedure Is-Safe(G, Color, Vertex, C, N)

for i ← 0 to N - 1 do

if G[Vertex][i] = 1 AND Color[i] = C then
return FALSE
end if

end for

return TRUE
```

Explanation of the Algorithm

Suppose a graph has **4 vertices** and we have **3 colors**:

M=3M=3

The algorithm works as follows:

Step 1 — Start with Vertex 0

Try Color 1.

```
Vertex 0 → Color 1
```

If it is safe, move to Vertex 1.

Step 2 — Color Vertex 1

Try Color 1.

If Vertex 0 and Vertex 1 are adjacent, they cannot have the same color.

So:

```
Color 1 → REJECT
```

Try Color 2:

```
Vertex 1 → Color 2
```

Then continue to the next vertex.

Step 3 — Continue Recursively

For every vertex:

```
Try Color 1
    ↓
Is it safe?
    ↓
  Yes → Assign color → Go to next vertex
    ↓
  No → Try next color
```

If none of the colors work, the algorithm goes back to the previous vertex and changes its color.

This is the **backtracking step**.

Safety Check

The `Is-Safe()` function is responsible for checking whether assigning color `C` to a vertex will violate the coloring constraint.

```
if G[Vertex][i] = 1 AND Color[i] = C
```

means:

- `G[Vertex][i] = 1` → Vertex and `i` are adjacent.
- `Color[i] = C` → The adjacent vertex already has the same color.

Therefore, the proposed color is invalid.

If such a vertex exists:

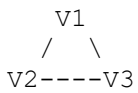
```
return FALSE
```

Otherwise:

```
return TRUE
```

Example

Consider the graph:



Suppose we have:

M=3M=3

Available colors: {1,2,3}\{1,2,3\}

A possible coloring is:

```
V1 → Color 1
V2 → Color 2
V3 → Color 3
```

Since every pair of connected vertices has different colors, the coloring is valid.

Backtracking Example

Suppose the algorithm reaches a situation like:

```
V1 → Color 1
V2 → Color 2
V3 → Color 1
V4 → No available color
```

Since Vertex 4 cannot be assigned any color, the algorithm backtracks:

```
Remove color from V3
Try another color for V3
```

If another color works:

```
V3 → Color 3
```

the algorithm proceeds again to Vertex 4.

Thus, invalid partial solutions are discarded instead of continuing unnecessarily.

Pruning

The main pruning condition is the safety check:

```
if Is-Safe(G, Color, Vertex, C, N) = FALSE
    do not assign C
```

This prevents the algorithm from exploring an entire subtree that already violates the coloring constraint.

Only **valid partial colorings** are extended further.

Correctness

The algorithm is complete because it tries every possible color from 11 to MM for each vertex, unless the color is immediately rejected as unsafe.

The algorithm guarantees that:

1. Every vertex receives a color.
2. A color is assigned only when no adjacent already-colored vertex has that color.
3. If a partial coloring cannot be completed, the algorithm backtracks.
4. If a valid MM-coloring exists, the algorithm will eventually find it.
5. If no valid coloring exists, the algorithm returns `FALSE`.

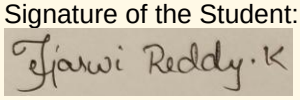
Therefore:

A coloring returned by the algorithm is always valid.

Code of Conduct

I Tejaswi Reddy K (192371036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:



RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Question No.	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)	Total Marks
Q1 (40 Marks)						
Q2 (30 Marks)						
Q3 (30 Marks)						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____